

Use Cases and Solutions for Multi-Agent Authorization

Chunchi Peter Liu, Yuan Ni

IETF 126, Vienna

- **Use Cases**
- **Cross-Domain Task Context Propagation**
- **Batch Authorization Delegation**

➤ Use Cases

➤ **Cross-Domain Task Context Propagation**

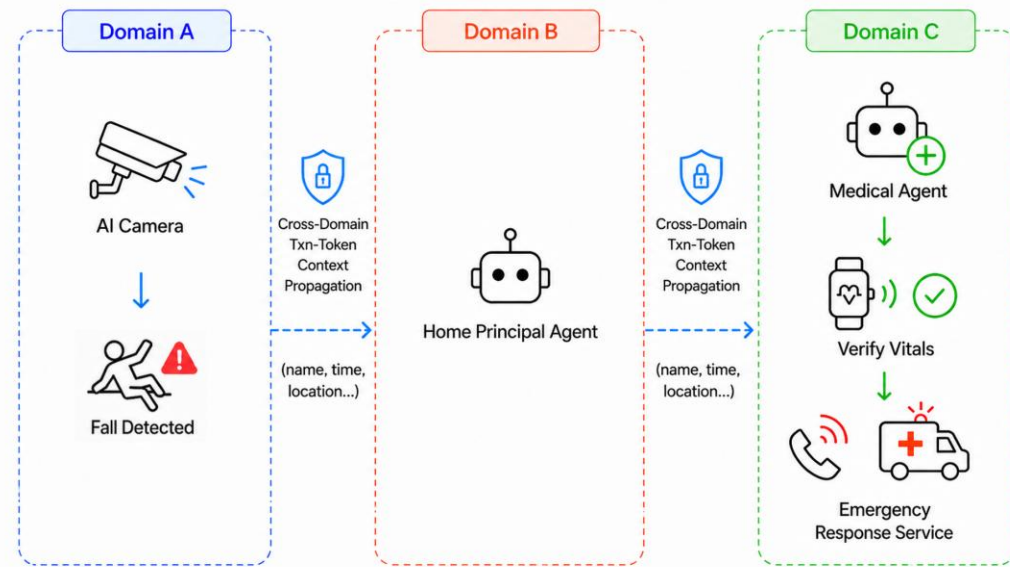
➤ **Batch Authorization Delegation**

Cross-Domain Task Context Propagation

Background:

- **Internet of Agents** presents a vision of an open ecosystem where agents and other resources (services, tools, etc.) cooperate across different domains. (Supporters: Anthropic, Cisco, Google, Huawei, Tencent.....)
- During this process, preserving **Cross-Domain Task Context during propagation** becomes necessary.

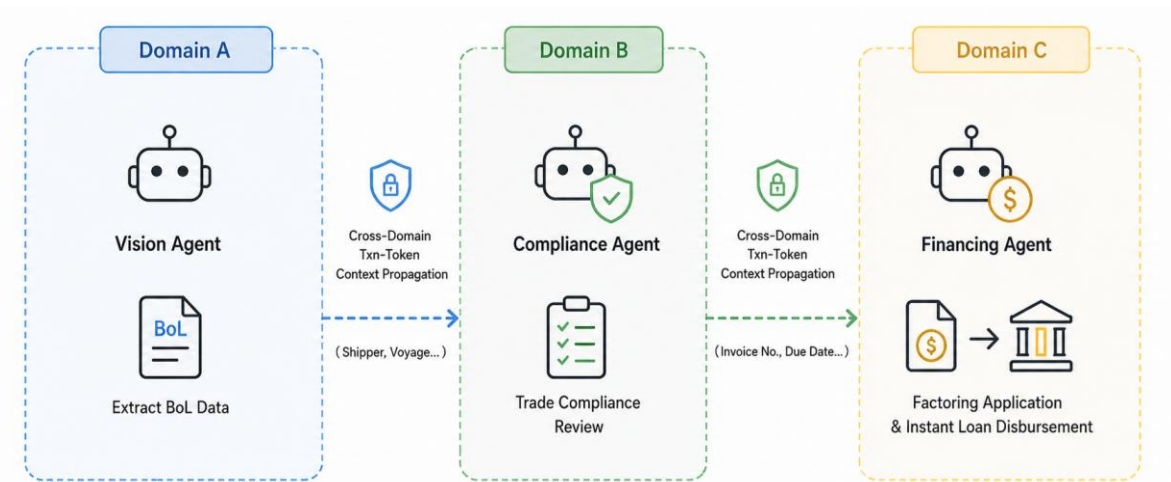
Use cases: both of the cases below need cross-domain resource collaboration.



Agent-Driven Emergency Response:

AI camera detects an elderly fall

- > **Home principal** agent assesses and invokes a **Medical Agent**
- > **Medical Agent** verifies vital signs via wearable devices and invokes an **Emergency Response Service**



Cross-Border Agent-Driven Supply Chain Finance:

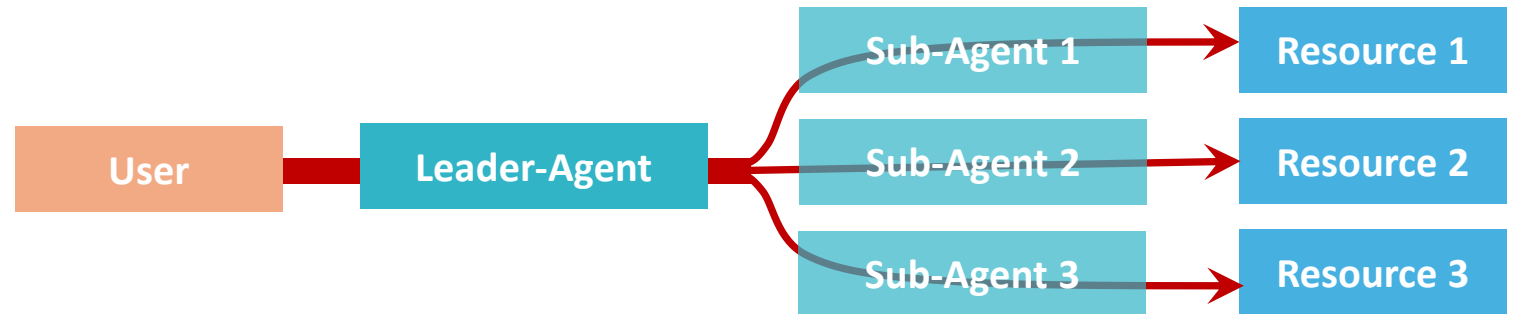
Vision Agent extracts BoL data

- > **Compliance Agent** runs trade compliance review
- > **Financing Agent** submits factoring application and executes instant loan disbursement.

Batch Authorization Delegation

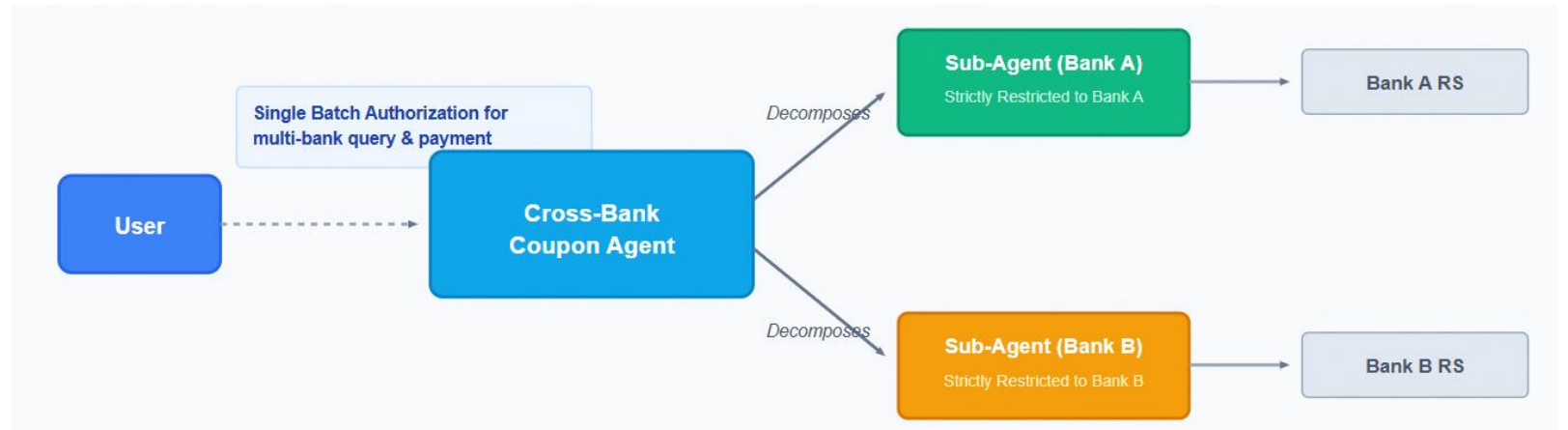
- ❑ **Scenario:** Since a task may consists of a batch of sub-tasks and requires a batch of authorization to different resources, a new resource access scenario appears:

A leader-agent (eg. a principal personal assistant AI Agent) coordinates multiple sub-agents to complete a task.



- ❑ **Use Case:**

A cross-bank Coupon Agent can coordinate multiple bank-specific sub-agents to calculate the best discount from all of the user's bank accounts for a single payment.



- ❑ **Core Challenge:**

One-Time Batch Authorization (to avoid repeated user interactions) + **Downscoped Delegation** (to ensure each sub-agent accesses ONLY its resource)

- **Use Cases**
- **Cross-Domain Task Context Propagation**
- **Batch Authorization Delegation**

Cross-Domain Task Context Propagation—Gap

Existing Building Blocks:

Drafts	What It Does	
Txn-Token [I-D.ietf-oauth-transaction-tokens]	Preserves user identity, workload identity, authz context (txn , tctx , rctx , req_wl) within a single domain	Complete data model defined.
Identity Chaining [I-D.ietf-oauth-identity-chaining]	Enables cross-domain identity and authorization propagation via JAGs	Cross-domain transport mechanism defined.

- ❑ **GAP:** Need design for Encapsulate Txn-Token contexts (**txn**, **tctx**, **rctx**, **req_wl**) into JAGs of Identity Chaining.
- ❑ Both blocks are mature; **ONLY** need to combine them!

Cross-Domain Task Context Propagation— Two solutions

Both works build on Identity Chaining + Txn-Token.

Drafts	<i>Txn-Token JAG Profile</i> <i>draft-fletcher-transaction-token-chaining-profile</i>	<i>Cross-Domain Txn Token (our draft)</i> <i>draft-liu-oauth-cross-domain-txn-token</i>
Flow	Txn-Token → JAG → Access Token (3 RTTs)	Mode A: Txn-Token 1 -> JAG -> Access Token -> Txn-Token 2 <i>Same as JAG Profile. Direct composition of Identity Chaining + Txn-Token. Minimal changes.</i>
		Mode B : Txn-Token 1 -> JAG -> Txn-Token 2 (1 RTTs) <i>JAG directly exchanged for Txn-Token. Saves 2 RTTs. Scalable. Requires extension that allow TTS to accept JAG as subject_token.</i>
Claims	- txn (pass, immutable) - tctx (do not pass) - rctx (pass, minimized) - req_wl (do not pass)	- txn (pass, immutable) - tctx (pass, minimized) - rctx (pass, minimized) - req_wl (pass, each hop appended, minimized) Data Minimization: AS MAY remove/redact claims (tctx, rctx, req_wl) not needed downstream; log locally if audit required.

- **Use Cases**
- **Cross-Domain Task Context Propagation**
- **Batch Authorization Delegation**

Batch Authorization Delegation—Gap

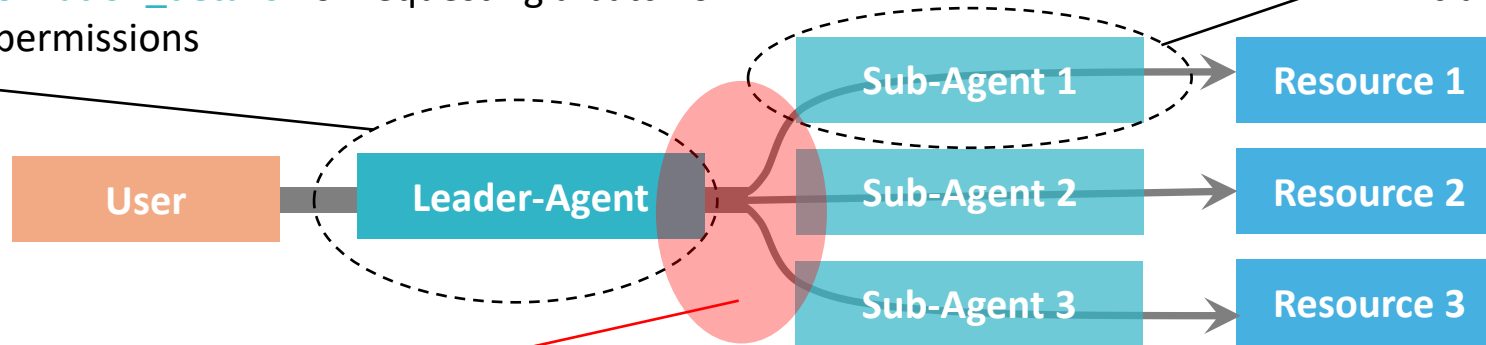
❑ What Existing OAuth Standards Provide?

OAuth 2.0 Rich Authorization Request (RAR) [RFC9396]

Defines [authorization_details](#) for requesting a batch of fine-grained permissions

OAuth 2.0 Token Exchange [RFC8693]

Defines the only actor via the [may_act](#) claim.



Gap:

- RAR enables [one-time batch authorization](#) of permissions, but **cannot bind the permissions to different designated actors**.
- Token Exchange enables [delegation](#), but **cannot express which subset of permissions is delegated to which actor**.

Batch Authorization Delegation—Our Solution

Add **may_act** Delegation for Each Downstream Sub-agent

OAuth 2.0 Token Exchange [RFC8693]	This draft
<pre>"aud": "https://consumer.example.com", "iss": "https://issuer.example.com", "sub": "user@example.com", "client_id": "leader_agent@example.com", "scope": "status feed", "may_act": { "sub": "admin@example.com" // A}</pre>	<pre>"aud": "https://issuer.example.com", "iss": "https://issuer.example.com", "sub": "user@example.com", "client_id": "leader_agent@example.com", "authorization_details": [{ "type": "flight_booking", "actions": ["search", "book"], "locations": ["https://example.com/flights"], "may_act": { "sub": "flight_agent@example.com" // A}}, { "type": "hotel_reservation", "actions": ["search", "book"], "locations": ["https://example.com/hotels"], "may_act": { "sub": "hotel_agent@example.com" // B}}]</pre>
Leader agent can only delegate the permission to A.	Leader agent can delegate one permission to A; and delegate another permission to B;